Informe de Análisis Estático de Seguridad (SAST)

Sistema de Producción e Inventario: Pollos Krosty · Laravel 11 / PHP

Herramienta: Semgrep OSS 1.178.0
Fecha del análisis: 29/09/2026
Rama analizada: `dev` (commit `8cf7ab2`)
Tipo de análisis: Estático, sin ejecutar la aplicación

1. Resumen ejecutivo

307	414	0	5
archivos analizados	reglas de seguridad aplicadas	vulnerabilidades detectadas	conjuntos de reglas (rulesets)

> **Resultado:** Semgrep **no encontró vulnerabilidades** de severidad ERROR, WARNING ni INFO en el código fuente del proyecto. Una revisión manual complementaria encontró **1 observación de configuración** y **1 de buenas prácticas**, ambas de riesgo bajo. También encontró riesgos de lógica de negocio, que ninguna herramienta SAST puede detectar (sección 6).

2. Alcance del análisis

Carpeta	Archivos	Contenido
`app/`	112	Controladores (web y API), modelos, servicios, middleware, Form Requests
`resources/`	124	Vistas Blade (119), JavaScript y CSS
`database/`	42	Migraciones, seeders y factories
`config/`	16	Configuración de la aplicación
`bootstrap/`, `routes/`, `public/index.php`, `.env.example`	13	Rutas web/API, arranque y registro de middleware

Excluidos: `vendor/` y `node_modules/` (dependencias de terceros), `storage/` y los archivos compilados. Por tipo: 178 PHP, 119 Blade, 3 JS, 2 CSS y 5 archivos de configuración.

3. Reglas aplicadas

Ruleset	Qué detecta
`p/php`	Inyección SQL, ejecución de comandos (`exec`, `eval`, `system`), deserialización insegura, inclusión de archivos
`p/security-audit`	Patrones inseguros generales: criptografía débil, SSRF, redirecciones abiertas, XSS

`p/owasp-top-ten`	Categorías OWASP Top 10: A01 Control de acceso, A03 Inyección, A05 Configuración, A07 Autenticación, etc.
`p/secrets`	Claves, contraseñas y tokens escritos en el código (API keys, credenciales)
`p/javascript`	XSS en el DOM (`innerHTML` , `eval`), criptografía débil en el frontend

Comando ejecutado:

```
semgrep scan --config p/php --config p/security-audit --config p/secrets --config p/javascript --conf
ig p/owasp-top-ten --metrics=off --no-git-ignore --json -o result.json app routes config resources da
tabase bootstrap public/index.php .env.example
```

4. Resultados de Semgrep

Severidad	Cantidad
ERROR (alta)	0
WARNING (media)	0
INFO (baja)	0

Avisos del motor: 4 "Internal matching error" en `resources/js/app.js` y `resources/js/crud-layout.js` . Vienen de reglas para las librerías *crypto-js*, *Express* y *Koa*, que este proyecto no usa. No son vulnerabilidades y no afectan el resultado. El reporte anterior de Semgrep del proyecto (`semgrep-report.json`) también daba 0 hallazgos, así que el resultado es consistente.

5. Verificación manual y controles de seguridad

Para confirmar que el 0 no es un falso negativo, se revisaron a mano los patrones de mayor riesgo:

Control / riesgo	Evidencia en el código	Estado
Inyección SQL (OWASP A03)	Se usa Eloquent/Query Builder. Las 3 consultas crudas usan *bindings* (`?`): `LoginRequest.php:67` , `ReglaConversion.php:109` , `ReporteController.php:87` (este último sin datos del usuario).	Protegido
XSS (OWASP A03)	Las vistas Blade usan `{{ }}` , que escapa automáticamente. No hay `{!! !!}` en ninguna vista.	Protegido
CSRF	Laravel 11 aplica el middleware `VerifyCsrfToken` a todas las rutas web y no hay excepciones configuradas.	Protegido
Autenticación (OWASP A07)	Las rutas web usan el middleware `auth` . La API usa `auth:sanctum` (tokens). El login tiene límite de intentos (`throttle:5,1`).	Protegido
Control de acceso por rol (OWASP A01)	El middleware propio `CheckRoutePermission` (`route.permission`) valida los permisos del rol en cada ruta.	Protegido
Datos sensibles en caché	El middleware `NoStoreSensitive` envía cabeceras *no-store* en los reportes y los datos sensibles.	Protegido

Secretos en el repositorio	`.env` está en `.gitignore` y no está versionado. `p/secrets` no encontró credenciales en el código.	Protegido
Validación de entrada	Se usan Form Requests (`VentaRequest` , `IngresoInsumoRequest` , `MermaRequest` , etc.) y `$request->validated()` .	Protegido
Integridad de datos	Las operaciones de stock usan `DB::transaction` . La producción usa `lockForUpdate()` para evitar condiciones de carrera.	Protegido

Observaciones de la revisión manual

#	Observación	Riesgo	Recomendación
1	`APP_DEBUG=true` en `.env` y `.env.example` (OWASP A05). En producción mostraría trazas de error con rutas y datos internos.	Bajo solo si se despliega así	En producción usar `APP_ENV=production` y `APP_DEBUG=false` .
2	`ComponenteProductoController` (store/update, líneas 40 y 57) usa `$request->all()` . Como `estado` está en `$fillable` , un usuario podría enviar ese campo aunque no esté en el formulario (asignación masiva).	Bajo	Reemplazar por `$request->validate([...])` y usar solo el arreglo validado.

6. Riesgos de lógica de negocio (fuera del alcance de SAST)

Semgrep detecta patrones de código inseguro, pero **no entiende las reglas del negocio**. En la revisión de las transacciones se identificaron estos puntos:

#	Hallazgo	Ubicación	Riesgo
1	El `total` de la venta viene del formulario y no se recalcula en el servidor sumando los `sub_total` . Una petición manipulada podría registrar un total distinto.	`VentaController::store`	Medio
2	Anular una venta (`destroy`) no devuelve el stock de los productos.	`VentaController::destroy`	Medio
3	La venta y la merma validan el stock sin `lockForUpdate()` . Con dos usuarios a la vez, el stock podría quedar en negativo.	`VentaController` , `MermaController`	Bajo
4	Editar o eliminar un ingreso de insumos resta stock sin comprobar si esos insumos ya se consumieron en producción.	`IngresoInsumoController::update/destroy`	Bajo

7. Conclusión

El análisis estático con Semgrep, usando 414 reglas de PHP, OWASP Top 10, auditoría general, JavaScript y secretos, **no encontró vulnerabilidades** en los 307 archivos del proyecto. Esto concuerda con las buenas prácticas del código: consultas parametrizadas con Eloquent, escape automático en Blade, protección

CSRF, autenticación con Sanctum, control de acceso por roles mediante middleware, límite de intentos en el login y transacciones con bloqueo pesimista.

Las mejoras recomendadas son configurar `APP_DEBUG=false` para producción, eliminar el uso de `$request->all()` y reforzar la lógica de ventas (recalcular el total en el servidor y devolver el stock al anular).

Informe generado a partir de la salida JSON de Semgrep OSS 1.178.0 y de la revisión manual del código fuente.